M12 # Hooks : Patterns Concrets

Méthodologie Intermediate

Exemples pratiques de hooks pour automatiser le workflow

Auto-format après édition

`PostToolUse`, matcher `Edit|Write` : choisir le formateur par extension.

```
INPUT=$(cat)
FILE=$(echo "$INPUT" | jq -r '.tool_input.file_path')
EXT="${FILE##*.}"
case "$EXT" in
  ts|tsx) npx prettier --write "$FILE" 2>/dev/null ;;
  py)     black "$FILE" 2>/dev/null ;;
  go)     gofmt -w "$FILE" 2>/dev/null ;;
  rs)     rustfmt "$FILE" 2>/dev/null ;;
esac
exit 0
```

Attendre la fin du formatage avant les tests ou une nouvelle écriture : l'exécution en arrière-plan crée un risque de concurrence.

Notification macOS à la fin

`Stop` : afficher une notification et jouer un son.

```
INPUT=$(cat)
MESSAGE=$(echo "$INPUT" | jq -r '.last_assistant_message // "Done"')
osascript - "$MESSAGE" <<'APPLESCRIPT'
on run argv
  display notification (item 1 of argv) with title "Claude Code"
end run
APPLESCRIPT
afplay /System/Library/Sounds/Hero.aiff &
exit 0
```

Sécurité : bloquer les commandes dangereuses

`PreToolUse`, matcher `Bash` : bloquer avant l'exécution.

```
COMMAND=$(cat | jq -r '.tool_input.command')
DANGEROUS=("rm -rf /" "dd if=" "DROP DATABASE"
           "git push --force.*main")
for pattern in "${DANGEROUS[@]}"; do
  if [[ "$COMMAND" =~ $pattern ]]; then
    echo "BLOCKED: $pattern" >&2
    exit 2
  fi
done
exit 0
```

Exit 2 bloque cette action. La liste reste contournable ; compléter avec permissions et isolation.

Git auto-stage après édition

`PostToolUse`, matcher `Edit|Write` : indexer le fichier modifié.

```
INPUT=$(cat)
FILE=$(echo "$INPUT" | jq -r '.tool_input.file_path // empty')
[[ -z "$FILE" || "$FILE" == "null" ]] && exit 0
git add -- "$FILE" 2>/dev/null
exit 0
```

RTK auto-wrapper

`PreToolUse` sur `Bash` : pour `git log`, `cargo test`, `pnpm list`, suggérer `rtk <commande>` via le JSON `hookSpecificOutput.additionalContext`, ou utiliser un wrapper dédié. Une sortie stdout brute ne suffit pas à informer Claude.

Logger toutes les commandes exécutées

`PostToolUse`, matcher `Bash` : journaliser les commandes.

```
COMMAND=$(cat | jq -r '.tool_input.command // empty')
mkdir -p ~/.claude/logs
printf '%s | %s\n' "$(date -u +%FT%T)" "$COMMAND" \
  >> ~/.claude/logs/commands.log
exit 0
```

Pattern dispatcher : un seul point d'entrée

Dans `settings.json`, un matcher `Edit|Write|Bash` appelle `dispatch.sh`, qui choisit un handler dans `.claude/hooks/handlers/` selon l'outil ou le fichier.

Remplacement de sortie d'outil (v2.1.121)

`PostToolUse` peut remplacer la réponse de Bash, Read, Write, Edit ou MCP via `hookSpecificOutput.updatedToolOutput` : anonymisation, compression ou métadonnées.

```
jq 'select(.tool_name == "Bash") |
  {hookSpecificOutput: {hookEventName: "PostToolUse",
    updatedToolOutput: (.tool_response |
      .stdout |= gsub("ghp_[A-Za-z0-9]+"; "[REDACTED]") |
      .stderr |= gsub("ghp_[A-Za-z0-9]+"; "[REDACTED]"))}}'
```

Respecter le schéma réel de réponse. Le remplacement n'annule ni les effets de bord ni la télémétrie déjà capturée. Ce motif ne couvre pas tous les secrets.

Checklist installation

```
chmod +x .claude/hooks/mon-hook.sh
# Tester manuellement avant d'activer :
echo '{"tool_name":"Edit","tool_input":{"file_path":"test.ts"}}' \
  | .claude/hooks/mon-hook.sh
```